

Lecture 17 & 18 - Module 5.2 Training MLP

COMP 551 Applied machine learning

Yue Li

Assistant Professor

School of Computer Science

McGill University

March 13 & 18, 2025

Outline

Objectives

Backpropagation

- Training a 1-layer MLP using backpropagation

- Training a L-layer MLP using backpropagation

- Application to Iris flower classification

Automatic differentiation

Brief survey of techniques for training DNN

Summary

Learning objectives

Understanding the basic ideas of

- Backpropagation by chain rule to derive analytical gradients by hands
- Automatic differentiation
- Strategies in training neural networks in practice

Outline

Objectives

Backpropagation

- Training a 1-layer MLP using backpropagation

- Training a L-layer MLP using backpropagation

- Application to Iris flower classification

Automatic differentiation

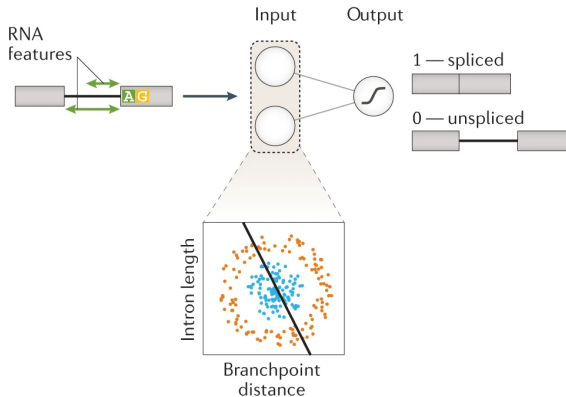
Brief survey of techniques for training DNN

Summary

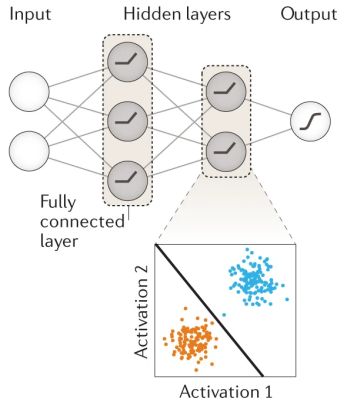
Multi-layer neural network

Neural networks can learn complex non-linear functions.

a Single-layer neural network (logistic regression)

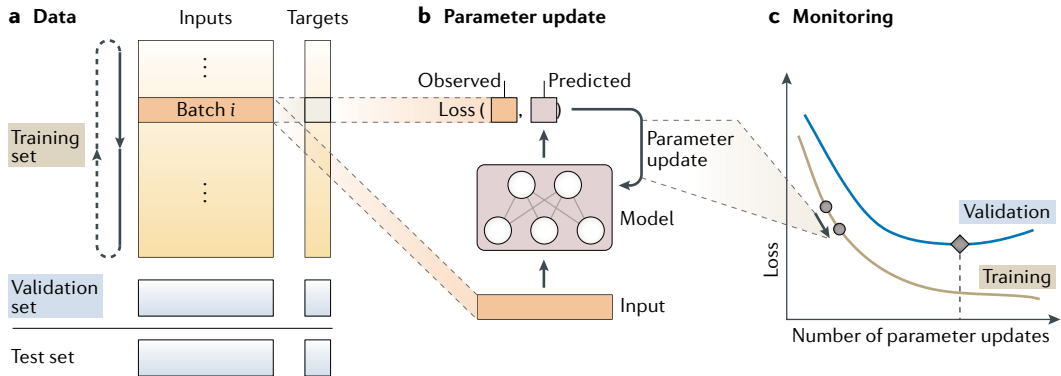


b Multilayer neural network



Gökçen Eraslan, Ziga Avsec, Julien Gagneur, and Fabian J Theis. Deep learning: new computational modelling techniques for genomics. *Nature Reviews Genetics*, 20(7):389-403, 2019.

Training neural networks via stochastic gradient descent



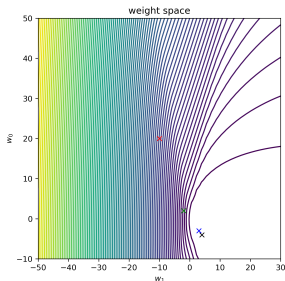
Gökçen Eraslan, Ziga Avsec, Julien Gagneur, and Fabian J Theis. Deep learning: new computational modelling techniques for genomics. Nature Reviews Genetics, 20(7):389-403, 2019.

Cost landscape of neural network consists of many local optima

Cross-entropy of linear logistic regression:

$$\sum_n CE(y^{(n)}, \sigma(\mathbf{x}^{(n)} \mathbf{w}))$$

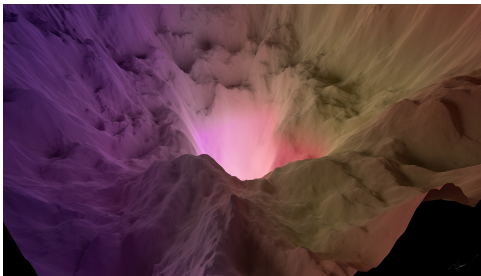
CE contour of logistic regression with two coefficients (Module 4.2):



CE of neural network:

$$\sum_n CE(y^{(n)}, \sigma(f(\dots, f(\mathbf{x}^{(n)}; \mathbf{W}^{\{1\}}); \dots; \mathbf{W}^{\{L\}}); \mathbf{w}^{\{o\}}))$$

One million loss values for a convolution neural network trained on ImageNet Dataset. Each loss value in the landscape corresponds to one set of network weights:



Objective: update the nnet's weights $\mathbf{W}^{\{\ell\}}$ at each layer using GD

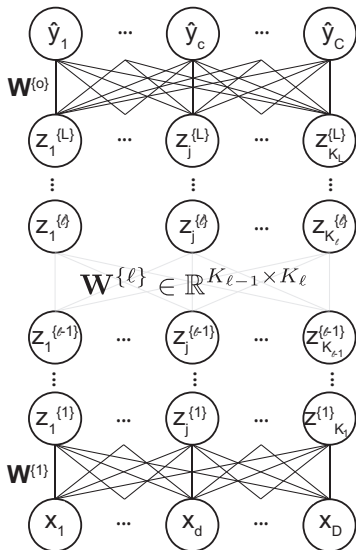
Recall in Module 4.4: gradient descent (GD) on a *vector* of coefficients $\mathbf{w}$

Algorithm 1 GradientDecent(α)

- 1: **while** not converged **do**
 - 2: $\mathbf{w}^{\{t+1\}} \leftarrow \mathbf{w}^{\{t\}} - \alpha \nabla J(\mathbf{w}^{\{t\}})$
 - 3: **end while**
-

where

- $J(\mathbf{w})$ is the loss given weights $\mathbf{w}$
- $\mathbf{w}^{\{t\}}$: weights at the t^{th} iteration
- $\nabla_{\mathbf{w}} f(\mathbf{w}) = \left[\frac{\partial}{\partial w_1} J(\mathbf{w}), \dots, \frac{\partial}{\partial w_D} J(\mathbf{w}) \right]^\top$
are the gradients



Jacobian *matrix* contains *all* partial derivatives w.r.t. $\mathbf{W}^{\{\ell\}}$ at each layer ℓ

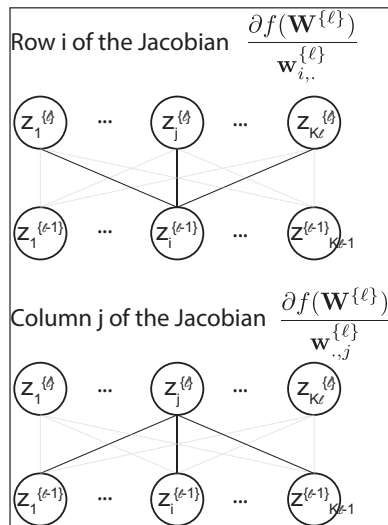
- $f(\mathbf{W}) : \mathbb{R}^{K_{\ell-1}} \rightarrow \mathbb{R}^{K_{\ell}}$:

$$\mathbf{z}_{1 \times K_{\ell}}^{\{\ell\}} = \phi(\mathbf{z}_{1 \times K_{\ell-1}}^{\{\ell-1\}} \mathbf{W}_{K_{\ell-1} \times K_{\ell}}^{\{\ell\}})$$

- *Jacobian* of all partial derivatives:

$$\mathbf{J}_{f(\mathbf{W}^{\{\ell\}})} = \begin{bmatrix} \frac{\partial f(\mathbf{W}^{\{\ell\}})}{w_{1,1}^{\{\ell\}}} & \dots & \frac{\partial f(\mathbf{W}^{\{\ell\}})}{w_{1,K_{\ell}}^{\{\ell\}}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f(\mathbf{W}^{\{\ell\}})}{w_{K_{\ell-1},1}^{\{\ell\}}} & \dots & \frac{\partial f(\mathbf{W}^{\{\ell\}})}{w_{K_{\ell-1},K_{\ell}}^{\{\ell\}}} \end{bmatrix}$$

- $\partial f(\mathbf{W}^{\{\ell\}})/\partial \mathbf{w}_{i,\cdot}^{\{\ell\}}$: gradients of weights for input $z_i^{\{\ell-1\}}$
- $\partial f(\mathbf{W}^{\{\ell\}})/\partial \mathbf{w}_{\cdot,j}^{\{\ell\}}$: gradients of weights for output $z_j^{\{\ell\}}$



Recall the chain rules that we have used for learning logistic models

For binary logistic regression (Module 4.2):

$$L(\mathbf{w}) = -y \log(\hat{y}) - (1-y) \log(1-\hat{y}), \quad \text{where} \quad \hat{y} = \frac{1}{1 + \exp(-a)} \quad \text{and} \quad a = \sum_d x_d w_d$$

Using **chain rule**, we derived the partial derivative w.r.t. w_d :

$$\frac{\partial L(\mathbf{w})}{\partial w_d} = \frac{\partial L(\mathbf{w})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a} \frac{\partial a}{\partial w_d}$$

For multiclass logistic regression (Module 4.3):

$$L(\mathbf{W}) = - \sum_{c=1}^C y_c \log \hat{y}_c, \quad \text{where} \quad \hat{y}_c = \frac{\exp(a_c)}{\sum_c \exp(a_c)} \quad \text{and} \quad a_c = \sum_d x_d w_{d,c}$$

Using **chain rule**, we derived the partial derivative w.r.t. $w_{d,j}$ for class j and feature d :

$$\frac{\partial L(\mathbf{W})}{\partial w_{d,j}} = \sum_c \frac{\partial L_c}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial a_j} \frac{\partial a_j}{\partial w_{d,j}}, \quad \text{where} \quad L_c = -y_c \log \hat{y}_c$$

The same principle can be used to update the weights of a neural net.

Let's consider a 1-layer fully connected MLP for multiclass prediction

Forward pass

For each hidden unit $j \in \{1, \dots, K\}$:

$$a_j = \sum_d x_d w_{d,j}^{\{1\}} = \mathbf{x} \mathbf{w}_{\cdot j}^{\{1\}}, \quad z_j = \phi(a_j)$$

Or for all K hidden units in matrix form:

$$\mathbf{a} = \mathbf{x} \mathbf{W}^{\{1\}}, \quad \mathbf{z} = \phi(\mathbf{a})$$

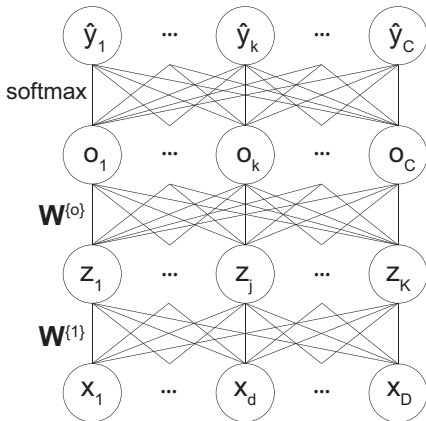
Output for class c :

$$o_c = \sum_j z_j w_{j,c}^{\{o\}} = \mathbf{z} \mathbf{w}_c^{\{o\}} \quad \text{or} \quad \mathbf{o} = \mathbf{z} \mathbf{W}^{\{o\}}$$

$$\hat{y}_c = \frac{\exp(o_c)}{\sum_c \exp(o_c)} \quad \text{or} \quad \hat{\mathbf{y}} = \frac{\exp(\mathbf{o})}{\sum_c \exp(o_c)}$$

Loss

$$L_c = -y_c \log \hat{y}_c, \quad L = \sum_c L_c$$



Backpropagating error derivatives from output layer to hidden layer

$$\frac{\partial L}{\partial w_{j,k}^{\{o\}}} = \sum_c \frac{\partial L_c}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial o_k} \frac{\partial o_k}{\partial w_{j,k}^{\{o\}}} = (\hat{y}_k - y_k) z_j$$

$$\frac{\partial L_c}{\partial \hat{y}_c} = -\frac{y_c}{\hat{y}_c}$$

$$\frac{\partial \hat{y}_c}{\partial o_k} = \hat{y}_c (\mathbb{I}[k = c] - \hat{y}_k)$$

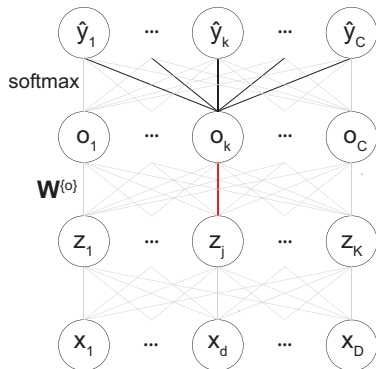
$$\frac{\partial o_k}{\partial w_{j,k}^{\{o\}}} = \frac{\partial}{\partial w_{j,k}^{\{o\}}} \sum_d z_j w_{j,k}^{\{o\}} = z_j$$

This is identical to the Multiclass Regression (Eq 4, Module 4.3) except replacing the inputs x_d 's with the hidden units z_j 's.

Recall we need to sum over all classes because softmax involves $w_{j,c}$ in predicting every class.

The Jacobian matrix for $\mathbf{W}^{\{o\}} \in \mathbb{R}^{K \times C}$:

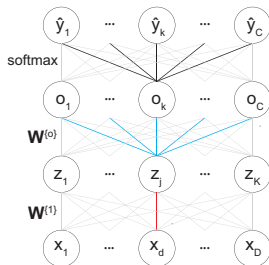
$$\frac{\partial L(\mathbf{W}^{\{o\}})}{\partial \mathbf{W}^{\{o\}}} = \frac{\partial \mathbf{o}}{\partial \mathbf{W}^{\{o\}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{o}} \frac{\partial L}{\partial \hat{\mathbf{y}}} = \mathbf{z}^\top (\hat{\mathbf{y}} - \mathbf{y})$$



Backpropagating error derivatives from hidden layer to input

$$\begin{aligned}
 \frac{\partial L}{\partial w_{d,j}^{\{1\}}} &= \frac{\partial L}{\partial z_j} \frac{\partial z_j}{\partial a_j} \frac{\partial a_j}{\partial w_{d,j}^{\{1\}}} \\
 &= \left(\sum_{k=1}^C \underbrace{\sum_{c=1}^C \frac{\partial L_c}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial o_k} \frac{\partial o_k}{\partial z_j}}_{\text{shown in prev. slide}} \right) \frac{\partial z_j}{\partial a_j} \frac{\partial a_j}{\partial w_{d,j}^{\{1\}}} \\
 &= \left(\sum_k (\hat{y}_k - y_k) w_{j,k}^{\{o\}} \right) \frac{\partial \phi(a_j)}{\partial a_j} x_d \\
 \frac{\partial o_k}{\partial z_j} &= \frac{\partial}{\partial z_j} \sum_j w_{j,k}^{\{o\}} z_j = w_{j,k}^{\{o\}} \\
 \frac{\partial z_j}{\partial a_j} &= \frac{\partial \phi(a_j)}{\partial a_j} \\
 \frac{\partial a_j}{\partial w_{d,j}^{\{1\}}} &= \frac{\partial}{\partial w_{d,j}^{\{1\}}} \sum_d w_{d,j}^{\{1\}} x_d = x_d
 \end{aligned}$$

- (1) inner sum over all C outputs w.r.t. o_k because o_k participates in the softmax for every $\hat{y}_c$;
 (2) outer sum over all C outputs w.r.t. z_j because z_j participates in computing every output o_k .

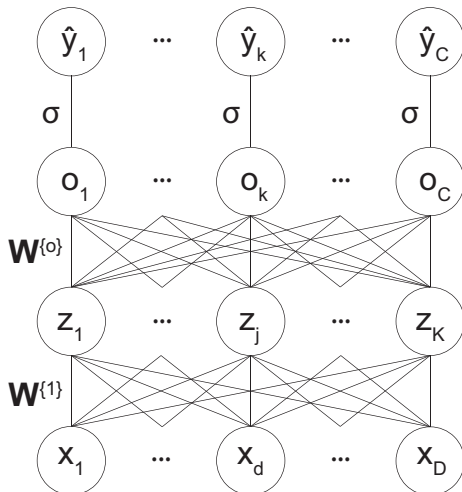


The Jacobian matrix for $\mathbf{W}^{\{1\}} \in \mathbb{R}^{D \times K}$:

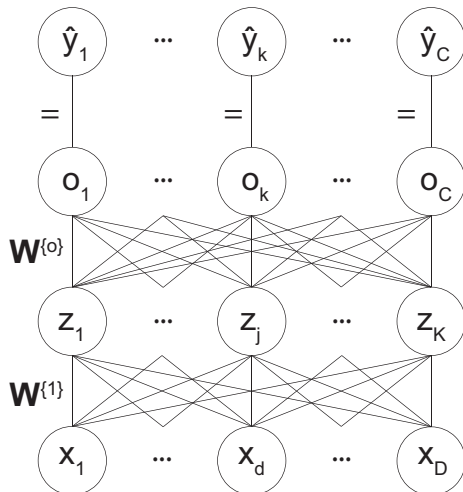
$$\frac{\partial L(\mathbf{W}^{\{1\}})}{\partial \mathbf{W}^{\{1\}}} = \mathbf{x}^\top \left((\hat{\mathbf{y}} - \mathbf{y})(\mathbf{W}^{\{o\}})^\top \odot \frac{\partial \mathbf{z}}{\partial \mathbf{a}} \right)$$

multi-task one-layer MLP

Multi-label binary classification



Multivariate regression



Gradients of multi-task one-layer MLP

Forward pass:

$$a_j = \sum_d x_d w_{d,j}^{\{1\}} = \mathbf{x} \mathbf{w}_{\cdot,j}^{\{1\}}$$

$$z_j = \phi(a_j)$$

$$o_c = \sum_c z_j w_{j,c}^{\{o\}}$$

Multi-label ($\neq$ C MLP- σ 's):

$$\hat{y}_c = \sigma(o_c)$$

$$L_{CE} = - \sum_c y_c \log \hat{y}_c - (1 - y_c) \log(1 - \hat{y}_c) = \sum_c L_c$$

$$\frac{\partial L_c}{\partial \hat{y}_c} = -\frac{y_c}{\hat{y}_c} + \frac{1 - y_c}{1 - \hat{y}_c} = \frac{\hat{y}_c - y_c}{\hat{y}_c(1 - \hat{y}_c)}$$

$$\frac{\partial \hat{y}_c}{\partial o_k} = \begin{cases} \hat{y}_c(1 - \hat{y}_c) & \text{if } k = c \\ 0 & \text{if } k \neq c \end{cases}$$

Backprop:

$$\frac{\partial L}{\partial w_{j,k}^{\{o\}}} = \sum_c \frac{\partial L_c}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial o_k} \frac{\partial o_k}{\partial w_{j,k}^{\{o\}}}$$

$$\frac{\partial L}{\partial w_{d,j}^{\{1\}}} = \left(\sum_k \sum_c \frac{\partial L_c}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial o_k} \frac{\partial o_k}{\partial z_j} \right) \frac{\partial z_j}{\partial a_j} \frac{\partial a_j}{\partial w_{d,j}^{\{1\}}}$$

Regression ($\neq$ C regression MLPs):

$$\hat{y}_c = o_c$$

$$L_{SSE} = \sum_c (y_c - \hat{y}_c)^2 = \sum_c L_c$$

$$\frac{\partial L_c}{\partial \hat{y}_c} = -2(y - \hat{y}_c)$$

$$\frac{\partial \hat{y}_c}{\partial o_k} = \mathbb{I}[k = c]$$

Gradients of 1-layer MLP for sigmoid and ReLU activation functions

Forward pass:

$$a_j = \sum_d x_d w_{d,j}^{\{1\}} = \mathbf{x} \mathbf{w}_{\cdot,j}^{\{1\}}$$

$$z_j = \phi(a_j)$$

$$o_c = \sum_j z_j w_{j,c}^{\{o\}}$$

Sigmoid activation:

$$z_j = \phi(a_j) \equiv \sigma(a_j) = \frac{1}{1 + \exp(-a_j)}$$

$$\frac{\partial z_j}{\partial a_j} = z_j(1 - z_j)$$

Backprop:

$$\frac{\partial L}{\partial w_{j,k}^{\{o\}}} = \sum_c \frac{\partial L_c}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial o_k} \frac{\partial o_k}{\partial w_{j,k}^{\{o\}}}$$

$$\frac{\partial L}{\partial w_{d,j}^{\{1\}}} = \left(\sum_k \sum_c \frac{\partial L_c}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial o_k} \frac{\partial o_k}{\partial z_j} \right) \frac{\partial z_j}{\partial a_j} \frac{\partial a_j}{\partial w_{d,j}^{\{1\}}}$$

ReLU activation:

$$z_j = \phi(a_j) \equiv \text{ReLU}(a_j) = \max(0, a_j)$$

$$\frac{\partial z_j}{\partial a_j} = \mathbb{I}[a_j > 0]$$

Outline

Objectives

Backpropagation

- Training a 1-layer MLP using backpropagation

- Training a L-layer MLP using backpropagation

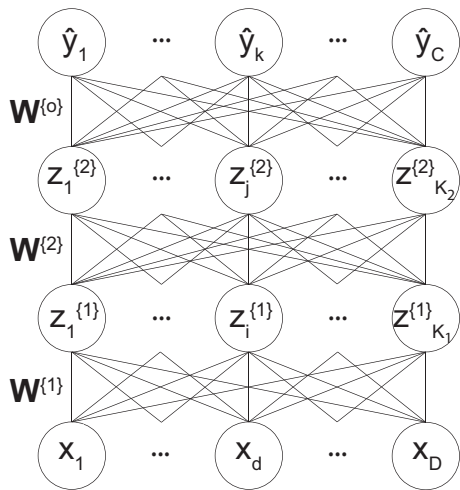
- Application to Iris flower classification

Automatic differentiation

Brief survey of techniques for training DNN

Summary

Two-layer MLP

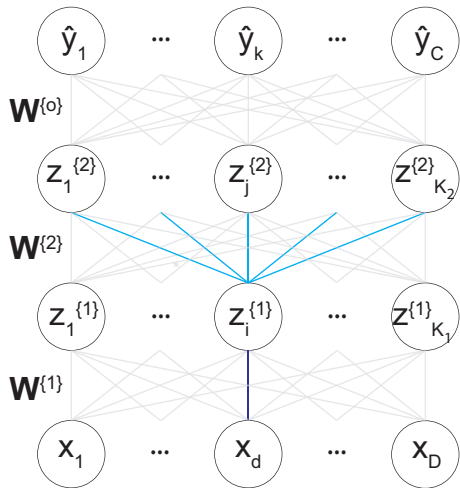


$$a_i^{\{1\}} = \sum_d x_d w_{d,i}^{\{1\}} = \mathbf{x} \mathbf{w}_{:,i}^{\{1\}}, \quad z_i^{\{1\}} = \phi(a_i^{\{1\}})$$

$$a_j^{\{2\}} = \sum_i z_i w_{i,j}^{\{2\}} = \mathbf{z}^{\{1\}} \mathbf{w}_{:,j}^{\{2\}}, \quad z_j^{\{2\}} = \phi(a_j^{\{2\}})$$

$$o_c = \sum_j z_j^{\{2\}} w_{j,c}^{\{o\}} = \mathbf{z}^{\{2\}} \mathbf{w}_{:,c}^{\{o\}}, \quad \hat{y}_c = f(o_c)$$

Backprop from hidden layer 1 to input layer



$$\frac{\partial L(\mathbf{W}^{(o)})}{\partial w_{j,k}^{(o)}} = \sum_c \frac{\partial L_c}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial o_k} \frac{\partial o_k}{\partial w_{j,k}^{(o)}}$$

$$\frac{\partial L(\mathbf{W}^{(2)})}{\partial w_{i,j}^{(2)}} = \underbrace{\left(\sum_k \sum_c \frac{\partial L_c}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial o_k} \frac{\partial o_k}{\partial z_j^{(2)}} \right)}_{\frac{\partial L}{\partial z_j^{(2)}}} \frac{\partial z_j^{(2)}}{\partial a_j^{(2)}} \frac{\partial a_j^{(2)}}{\partial w_{i,j}^{(2)}}$$

$$\frac{\partial L(\mathbf{W}^{(1)})}{\partial w_{d,i}^{(1)}} = \left(\sum_j \frac{\partial L}{\partial z_j^{(2)}} \frac{\partial z_j^{(2)}}{\partial a_j^{(2)}} \frac{\partial a_j^{(2)}}{\partial z_i^{(1)}} \right) \frac{\partial z_i^{(1)}}{\partial a_i^{(1)}} \frac{\partial a_i^{(1)}}{\partial w_{d,i}^{(1)}}$$

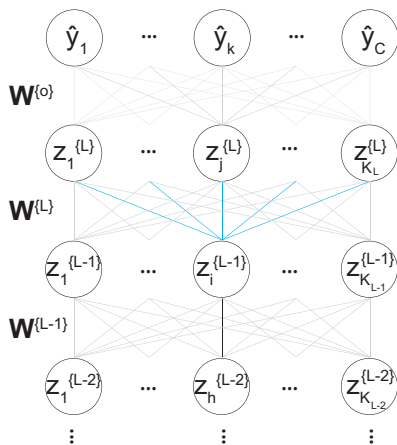
L-layer MLP: backprop to update layer $\mathbf{W}^{\{L-1\}}$

Forward pass:

$$a_i^{\{1\}} = \mathbf{x} \mathbf{w}_{\cdot,i}^{\{1\}}, \quad z_i^{\{1\}} = \phi(a_i^{\{1\}})$$

$$a_j^{\{\ell\}} = \mathbf{z}^{\{\ell-1\}} \mathbf{w}_{\cdot,j}^{\{\ell\}}, \quad z_j^{\{\ell\}} = \phi(a_j^{\{\ell\}}) \quad \text{for } \ell \in \{1, \dots, L\}$$

$$o_c = \mathbf{z}^{\{L\}} \mathbf{w}_{\cdot,c}^{\{o\}}, \quad \hat{y}_c = f(o_c)$$



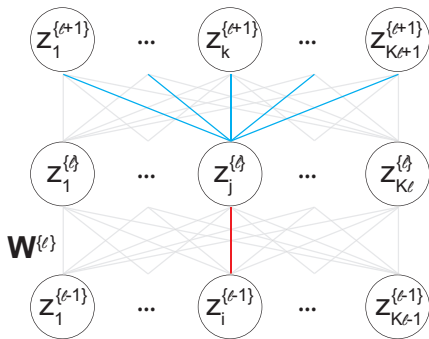
Backprop:

$$\frac{\partial L(\mathbf{W}^{\{o\}})}{\partial w_{j,k}^{\{o\}}} = \sum_{c=1}^C \frac{\partial L_c}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial o_k} \frac{\partial o_k}{\partial w_{j,k}^{\{o\}}}$$

$$\frac{\partial L(\mathbf{W}^{\{L\}})}{\partial w_{i,j}^{\{L\}}} = \underbrace{\left(\sum_{k=1}^C \sum_{c=1}^C \frac{\partial L_c}{\partial \hat{y}_c} \frac{\partial \hat{y}_c}{\partial o_k} \frac{\partial o_k}{\partial z_j^{\{L\}}} \right)}_{\frac{\partial L}{\partial z_j^{\{L\}}}} \frac{\partial z_j^{\{L\}}}{\partial a_j^{\{L\}}} \frac{\partial a_j^{\{L\}}}{\partial w_{i,j}^{\{L\}}}$$

$$\frac{\partial L(\mathbf{W}^{\{L-1\}})}{\partial w_{h,i}^{\{L-1\}}} = \left(\sum_{j=1}^{K_L} \frac{\partial L}{\partial z_j^{\{L\}}} \frac{\partial z_j^{\{L\}}}{\partial a_j^{\{L\}}} \frac{\partial a_j^{\{L\}}}{\partial z_i^{\{L-1\}}} \right) \frac{\partial z_i^{\{L-1\}}}{\partial a_i^{\{L-1\}}} \frac{\partial a_i^{\{L-1\}}}{\partial w_{h,i}^{\{L-1\}}}$$

Backprop from $\ell + 1$ to update $\mathbf{W}^{\{\ell\}}$



$$\frac{\partial L(\mathbf{W}^{\{\ell\}})}{\partial w_{i,j}^{\{\ell\}}} = \underbrace{\left(\sum_{k=1}^{K_{\ell+1}} \frac{\partial L}{\partial z_k^{\{\ell+1\}}} \frac{\partial z_k^{\{\ell+1\}}}{\partial a_k^{\{\ell+1\}}} \frac{\partial a_k^{\{\ell+1\}}}{\partial z_j^{\{\ell\}}} \right)}_{\frac{\partial L}{\partial z_j^{\{\ell\}}}} \frac{\partial z_j^{\{\ell\}}}{\partial a_j^{\{\ell\}}} \frac{\partial a_j^{\{\ell\}}}{\partial w_{i,j}^{\{\ell\}}}$$

Outline

Objectives

Backpropagation

- Training a 1-layer MLP using backpropagation

- Training a L-layer MLP using backpropagation

- Application to Iris flower classification

Automatic differentiation

Brief survey of techniques for training DNN

Summary

Implementation of a 1-layer MLP for *binary classification* (colab)

As a start, let's implement a 1-layer MLP with sigmoid activation to distinguish flowers in class 1 from flowers in class 2 only (i.e., a binary classification task).

```
def gradient(x, y, params):  
    v, w = params  
    z = logistic(np.dot(x, v)) #N x K  
    yh = logistic(np.dot(z, w)) #N  
    dy = yh - y #N  
    dw = np.dot(z.T, dy)/N #K  
    dz = np.outer(dy, w) #N x K  
    dv =  
    np.dot(x.T,  
           z * (1 - z) * dz)/N #D x K  
    dparams = [dv, dw]  
    return dparams
```

- $\mathbf{y} \in \{0, 1\}^{N \times 1}$, $\mathbf{X} \in \mathbb{R}^{N \times D}$
- $\mathbf{V} \in \mathbb{R}^{D \times K}$, $\mathbf{w} \in \mathbb{R}^{K \times 1}$
- $\mathbf{Z} \in [0, 1]^{N \times K}$

The gradient for $\mathbf{w} \equiv \mathbf{w}^{\{o\}} \in \mathbb{R}^{K \times 1}$:

$$\frac{\partial L}{\partial \mathbf{w}^{\{o\}}} = \sum_n \frac{\partial L}{\partial \hat{y}^{(n)}} \frac{\partial \hat{y}^{(n)}}{\partial o^{(n)}} \frac{\partial o^{(n)}}{\partial \mathbf{w}^{\{o\}}} = \mathbf{Z}^\top (\hat{\mathbf{y}} - \mathbf{y})$$

The Jacobian for $\mathbf{V} \equiv \mathbf{W}^{\{1\}} \in \mathbb{R}^{D \times K}$:

$$\frac{\partial L}{\partial \mathbf{z}^{(n)}} = \frac{\partial L}{\partial \hat{y}^{(n)}} \frac{\partial \hat{y}^{(n)}}{\partial o^{(n)}} \frac{\partial o^{(n)}}{\partial \mathbf{z}^{(n)}} = (\hat{y}^{(n)} - y^{(n)}) (\mathbf{w}^{\{o\}})^\top$$

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{v}_d} &= \sum_n \frac{\partial L}{\partial \mathbf{z}^{(n)}} \frac{\partial \mathbf{z}^{(n)}}{\partial \mathbf{a}^{(n)}} \frac{\partial \mathbf{a}^{(n)}}{\partial \mathbf{v}_d} \\ &= \mathbf{x}_d^\top \left(\mathbf{Z} \odot (1 - \mathbf{Z}) \odot \left\{ \frac{\partial L}{\partial \mathbf{z}^{(n)}} \right\}_{n=1}^N \right) \end{aligned}$$

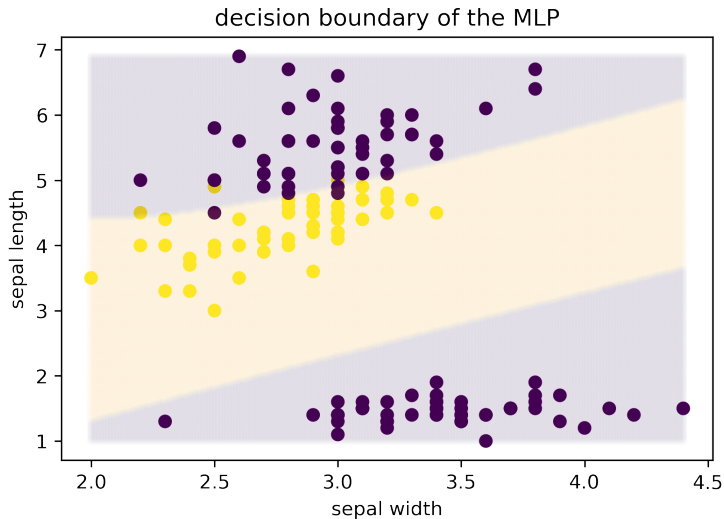
Training MLP with GD (colab)

Algorithm 2 MLP_fit_by_GD(α)

```
1: while  $\max(\|\nabla \mathbf{w}\|_2^2, \|\nabla \mathbf{V}\|_2^2) > \epsilon$  and  $t < \text{max\_iter}$  do  
2:    $\mathbf{w}^{\{t+1\}} \leftarrow \mathbf{w}^{\{t\}} - \alpha \nabla J(\mathbf{w}^{\{t\}})$   
3:    $\mathbf{V}^{\{t+1\}} \leftarrow \mathbf{V}^{\{t\}} - \alpha \nabla J(\mathbf{V}^{\{t\}})$   
4:    $t \leftarrow t + 1$   
5: end while
```

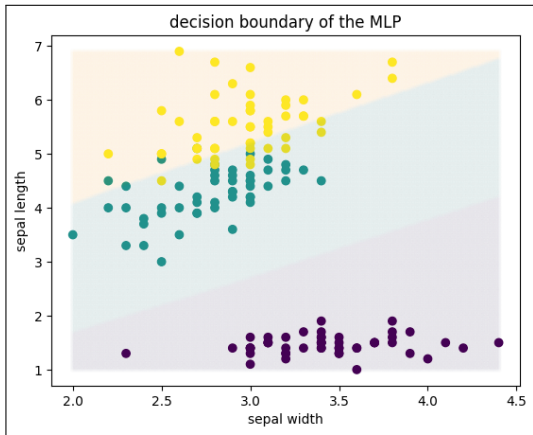
```
1 def fit(max_iters=1e4, epsilon=1e-8):  
2     while norm([dw,dv] > epsilon) and t < max_iters:  
3         dw, dv = gradient(x, y, w, v)  
4         w = w - lr*dw  
5         v = v - lr*dv  
6         t += 1
```

MLP's decision boundaries for Iris class 1 and 2 (colab)

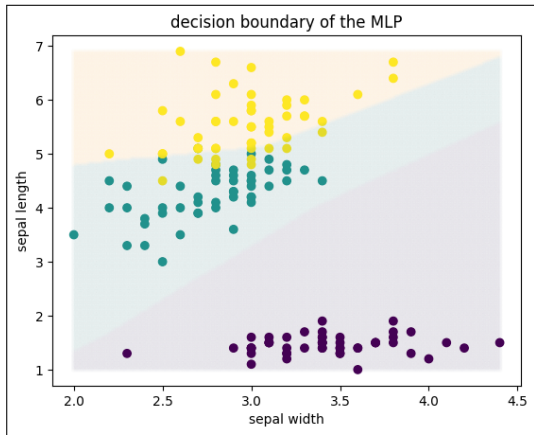


3-class L-layer MLP with softmax outputs and ReLU activation (colab)

1-layer MLP



2-layer MLP



Outline

Objectives

Backpropagation

- Training a 1-layer MLP using backpropagation

- Training a L-layer MLP using backpropagation

- Application to Iris flower classification

Automatic differentiation

Brief survey of techniques for training DNN

Summary

Methods to automate gradient computation

Deriving analytical gradients for training deep neural networks ($L > 2$) are tedious and repetitive. We can automate them in 3 ways.

1. Numerical approximation:

- Approximate gradient using small finite difference:

$$\frac{\partial}{\partial w_d} J(w_1, w_1, \dots, w_D) = \lim_{\epsilon \rightarrow 0} \frac{J(w_d + \epsilon, \mathbf{w}_{\setminus d}) - J(w_d, \mathbf{w}_{\setminus d})}{\epsilon}$$

- This needs multiple forward passes one per weight value – slow and inaccurate.
- It is useful to verify the gradient computation as we did in Module 4.2 and 4.3 and for getting gradient from non-differentiable “black-box” cost functions (out of scope of this course).

2. Symbolic differentiation

- manipulates mathematical expressions such that the derivative can be expressed by a set of pre-existing derivatives of known mathematic expressions (e.g., Mathematica).
- It is limited to the available rules.

3. Automatic differentiation (AutoDiff) (details follow)

- AutoDiff breaks down complex expressions down to *atomic operations*, for which the gradients are easy to compute.
- It then uses *chain rule* to compute the overall gradient – ideal for training neural networks via backpropagation.

AutoDiff builds a *computation graph* to differentiate a function

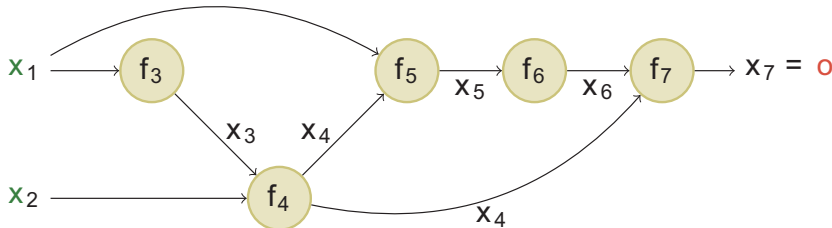
1. Break down an expression into atomic operations
2. Build a **directed acyclic graph (DAG)** with operations as internal nodes and input variables as leaf nodes

Forward mode (not used to train nnet)

- Start from the leaf nodes and propagate derivatives *forward*

Reverse mode (commonly used to train nnet, e.g., TensorFlow)

- 2.1 Bottom-up pass (forward pass): evaluate from the leaf nodes
- 2.2 Top-down pass (backprop): calculate the derivative by chain-rule



Example of AutoDiff in forward mode

Differentiating the function w.r.t. x_1 : $f(x_1, x_2) = x_2 \exp(x_1) \sqrt{x_1 + x_2 \exp(x_1)}$

Evaluation:

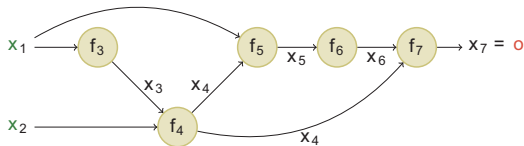
$$x_3 = f_3(x_1) = \exp(x_1)$$

$$x_4 = f_4(x_2, x_3) = x_2 x_3$$

$$x_5 = f_5(x_1, x_4) = x_1 + x_4$$

$$x_6 = f_6(x_5) = \sqrt{x_5}$$

$$x_7 = f_7(x_4, x_6) = x_4 x_6$$



Partial derivatives:

$$\frac{\partial x_1}{\partial x_1} = 1$$

$$\frac{\partial x_3}{\partial x_1} = \exp(x_1)$$

$$\frac{\partial x_4}{\partial x_1} = \frac{\partial x_2}{\partial x_1} x_3 + x_2 \frac{\partial x_3}{\partial x_1} = 0 x_3 + x_2 \exp(x_1) = x_2 \exp(x_1)$$

$$\frac{\partial x_5}{\partial x_1} = \frac{\partial x_1}{\partial x_1} + \frac{\partial x_4}{\partial x_1} = 1 + x_2 \exp(x_1)$$

$$\frac{\partial x_6}{\partial x_1} = 0.5 x_5^{-0.5} \frac{\partial x_5}{\partial x_1} = 0.5 x_5^{-0.5} (1 + x_2 \exp(x_1))$$

$$\begin{aligned} \frac{\partial x_7}{\partial x_1} &= \frac{\partial x_4}{\partial x_1} x_6 + x_4 \frac{\partial x_6}{\partial x_1} \\ &= x_2 \exp(x_1) x_6 + x_4 \left(0.5 x_5^{-0.5} (1 + x_2 \exp(x_1)) \right) \\ &= \left\{ x_4 (0.5 x_5^{-0.5}) + x_6 \right\} x_2 \exp(x_1) + x_4 (0.5 x_5^{-0.5}) \end{aligned}$$

Example of AutoDiff in reverse mode

Differentiating the function w.r.t. x_1 and x_2 : $f(x_1, x_2) = x_2 \exp(x_1) \sqrt{x_1 + x_2 \exp(x_1)}$

Forward pass:

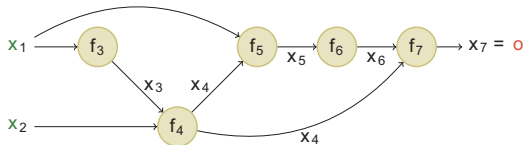
$$x_3 = f_3(x_1) = \exp(x_1)$$

$$x_4 = f_4(x_2, x_3) = x_2 x_3$$

$$x_5 = f_5(x_1, x_4) = x_1 + x_4$$

$$x_6 = f_6(x_5) = \sqrt{x_5}$$

$$x_7 = f_7(x_4, x_6) = x_4 x_6$$



Backward pass:

$$\frac{\partial o}{\partial x_7} = \frac{\partial x_7}{\partial x_7} = 1$$

$$\frac{\partial o}{\partial x_6} = \frac{\partial o}{\partial x_7} \frac{\partial x_7}{\partial x_6} = x_4$$

$$\frac{\partial o}{\partial x_5} = \frac{\partial o}{\partial x_6} \frac{\partial x_6}{\partial x_5} = x_4 (0.5 x_5^{-0.5})$$

$$\frac{\partial o}{\partial x_4} = \frac{\partial o}{\partial x_5} \frac{\partial x_5}{\partial x_4} + \frac{\partial o}{\partial x_7} \frac{\partial x_7}{\partial x_4} = x_4 (0.5 x_5^{-0.5}) + x_6$$

$$\frac{\partial o}{\partial x_3} = \frac{\partial o}{\partial x_4} \frac{\partial x_4}{\partial x_3} = \{x_4 (0.5 x_5^{-0.5}) + x_6\} x_2$$

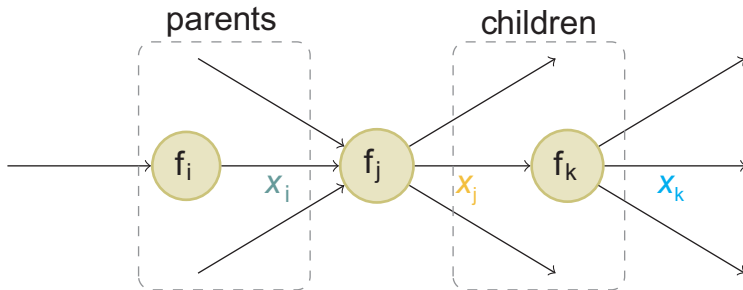
$$\frac{\partial o}{\partial x_2} = \frac{\partial o}{\partial x_4} \frac{\partial x_4}{\partial x_2} = \{x_4 (0.5 x_5^{-0.5}) + x_6\} x_3$$

$$\frac{\partial o}{\partial x_1} = \frac{\partial o}{\partial x_3} \frac{\partial x_3}{\partial x_1} + \frac{\partial o}{\partial x_5} \frac{\partial x_5}{\partial x_1}$$

$$= \{x_4 (0.5 x_5^{-0.5}) + x_6\} x_2 \exp(x_1) + x_4 (0.5 x_5^{-0.5})$$

General AutoDiff algorithm in reverse mode

$$\frac{\partial o}{\partial x_i} = \sum_{j \in \text{children}(i)} \frac{\partial o}{\partial x_j} \frac{\partial x_j}{\partial x_i}, \quad \frac{\partial o}{\partial x_j} = \sum_{k \in \text{children}(j)} \frac{\partial o}{\partial x_k} \frac{\partial x_k}{\partial x_j}$$



AutoDiff forward mode versus AutoDiff reverse mode

Forward mode:

e.g., $\frac{\partial w_d}{\partial w_d}$, $\frac{\partial \mathbf{xw}}{\partial w_d}$, $\frac{\partial \phi(\mathbf{xw})}{\partial w_d}$, $\frac{\partial \phi(\mathbf{xw})\mathbf{v}}{\partial w_d}$, $\frac{\partial (y - \phi(\mathbf{xw})\mathbf{v})^2}{\partial w_d}$

Pros:

- Natural to compute and easy to implement
- Memory efficient: we can discard the partial derivatives once we use them to compute the required derivatives on the forward-mode path

Cons:

- Inefficient to do when there are many parameters: we need to do a large number of forward-mode passes to compute the partial derivative for all of the parameters
 - one pass per parameter

Backward mode:

e.g., $\frac{\partial (y - \phi(\mathbf{xw})\mathbf{v})^2}{\partial \hat{y}}$, $\frac{\partial (y - \phi(\mathbf{xw})\mathbf{v})^2}{\partial \phi(\mathbf{xw})}$, $\frac{\partial (y - \phi(\mathbf{xw})\mathbf{v})^2}{\partial w}$

Pros:

- We store gradients for *all* parameters as we backprop
- This works well especially if we have a “pyramid” structured network (i.e., small number of outputs and large number of hidden and input units)
- For this reason, backward-mode AutoDiff is implemented in deep learning API such as Tensorflow, PyTorch, etc for training arbitrarily designed DNN

Cons:

- may take large ram to store all gradients

Computation graph for a 1-layer MLP with ℓ_2 weight penalty

AutoDiff works with any DAG. Consider the following MLP with ℓ_2 weight penalty:

$$\text{pre-activation} \quad \mathbf{z} = \mathbf{x}\mathbf{W}^{\{1\}}$$

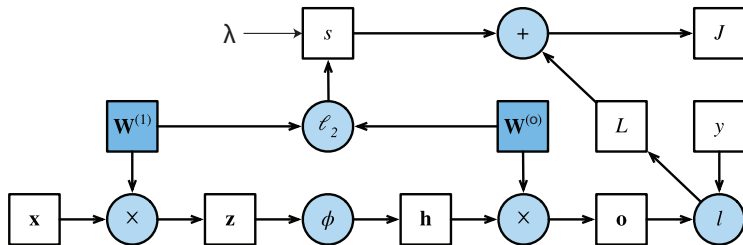
$$\text{activation} \quad \mathbf{h} = \phi(\mathbf{z})$$

$$\text{linear outputs} \quad \mathbf{o} = \mathbf{h}\mathbf{W}^{\{o\}}$$

$$\text{prediction loss} \quad L = \ell(\mathbf{o}, y)$$

$$\text{L2 penalty} \quad s = \lambda(\|\mathbf{W}^{\{1\}}\|_2^2 + \|\mathbf{W}^{\{o\}}\|_2^2)$$

$$\text{Total loss} \quad J = L + s$$



Outline

Objectives

Backpropagation

- Training a 1-layer MLP using backpropagation

- Training a L-layer MLP using backpropagation

- Application to Iris flower classification

Automatic differentiation

Brief survey of techniques for training DNN

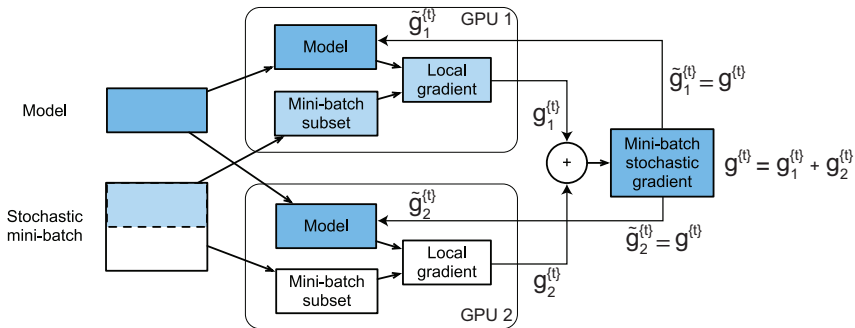
Summary

Large and deep neural networks are hard to train (or were to some people)

- **GPT-3**, or the third generation Generative Pre-trained Transformer, has over 175 billion parameters for natural language processing (NLP) tasks (i.e., language understanding; Module 5.4)! And the recent **GPT-4** has 1.76 trillion parameters!
- If GPT-3 is trained *in parallel* on 1000 GPUs using batch-size 1536, the time required to train this model is 34 days with cost of about \$5 M (next slide)
- Besides being computationally expensive to train a large and deep neural network, empirically established and ever-evolving computational techniques are also needed to achieve fast convergence and address **vanishing and exploding gradients**:
 1. Stochastic gradient (SGD) with adaptive learning rates (i.e., Adam; Module 4.4)
 2. Gradient clipping to address gradient explosion: (1) $\mathbf{g} = \min(1, \frac{1}{\|\mathbf{g}\|})\mathbf{g}$; (2) weight initialization from scaled variance $w \in \mathcal{N}(0, 2/(n_{in} + n_{out}))$, where n_{in} and n_{out} are incoming and outgoing edges from the current hidden unit (Murphy22 13.4.5.1)
 3. Modifying activation and architecture to address vanishing gradients (e.g., ResNet)
 4. Add regularization to address overfitting, e.g., weight decay, early stopping, dropout
 5. Bayesian optimization to choose network hyperparameters (last slide)

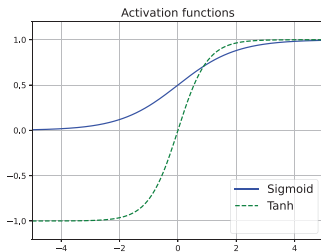
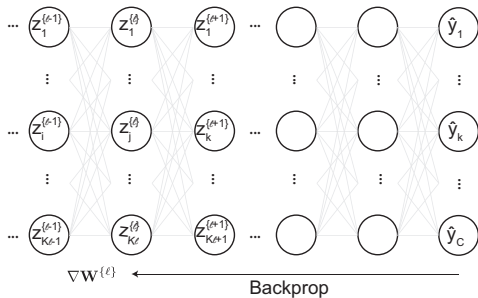
We briefly discuss some of the above points and will cover more in the following modules 5.3 and 5.4 for image and sequential data.

Parallel training to reduce training time (Murphy22 13.4.6)



1. At iteration t , partition the mini-batch across the K machines
2. Each machine k computes its own gradient $\mathbf{g}_k^{\{t\}}$
3. Collect all gradients at a central machine and sum them: $\mathbf{g}^{\{t\}} = \sum_{k=1}^K \mathbf{g}_k^{\{t\}}$
4. Broadcast the summed gradients back to all devices: $\tilde{\mathbf{g}}_k^{\{t\}} = \mathbf{g}^{\{t\}}$
5. Each machine updates its own copy of the parameters: $\boldsymbol{\theta}_k^{\{t\}} = \boldsymbol{\theta}_k^{\{t-1\}} - \alpha^{\{t\}} \tilde{\mathbf{g}}_k^{\{t\}}$

Vanishing gradient caused by multiplying small gradients during backprop



$$\frac{\partial L}{\partial w^{\{\ell\}}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z^{\{L\}}} \frac{\partial z^{\{L\}}}{\partial a^{\{L\}}} \frac{\partial a^{\{L\}}}{\partial z^{\{L-1\}}} \frac{\partial z^{\{L-1\}}}{\partial a^{\{L-1\}}} \frac{\partial a^{\{L-1\}}}{\partial z^{\{L-2\}}} \cdots \frac{\partial z^{\{\ell+1\}}}{\partial a^{\{\ell+1\}}} \frac{\partial a^{\{\ell+1\}}}{\partial z^{\{\ell\}}} \frac{\partial z^{\{\ell\}}}{\partial a^{\{\ell\}}} \frac{\partial a^{\{\ell\}}}{\partial w^{\{\ell\}}}$$

For sigmoid:

$$z = \sigma(a) = \frac{1}{1 + \exp(-a)}$$

$$\frac{\partial z}{\partial a} = z(1 - z) \in [0, 1]$$

For hyperbolic tangent:

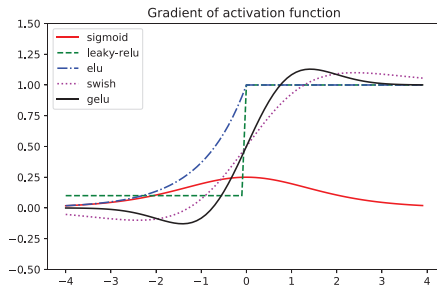
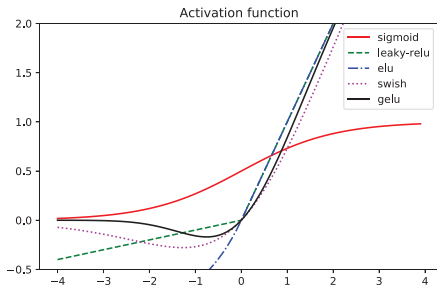
$$z = \tanh(a) = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)}$$

$$\frac{\partial z}{\partial a} = 1 - \tanh^2(a) \in [0, 1]$$

Non-saturating activation functions to reduce VG (Murphy22 13.4.3)

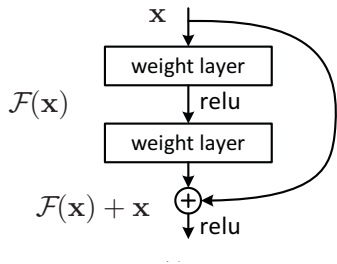
Name	Definition	Range	Reference
Sigmoid	$\sigma(a) = \frac{1}{1 + e^{-a}}$	$[0, 1]$	
Hyperbolic tangent	$\tanh(a) = 2 \sigma(2a) - 1$	$[-1, 1]$	
Softplus	$\sigma_+(a) = \log(1 + e^a)$	$[0, \infty]$	[GBB11]
Rectified linear unit	$\text{ReLU}(a) = \max(a, 0)$	$[0, \infty]$	[GBB11; KSH12]
Leaky ReLU	$\max(a, 0) + \alpha \min(a, 0)$	$[-\infty, \infty]$	[MHN13]
Exponential linear unit	$\max(a, 0) + \min(\alpha(e^a - 1), 0)$	$[-\infty, \infty]$	[CUH16]
Swish	$a\sigma(a)$	$[-\infty, \infty]$	[RZL17]
GELU	$a\Phi(a)$	$[-\infty, \infty]$	[HG16]

Can you compute derivative w.r.t. a for all of the above activation functions?



Residual connection mitigates vanishing gradient

Residual connection allows gradient to flow from the output layer to the early layer.



The residual connection allows gradient at layer ℓ directly depend on the gradient at layer L regardless of the network depth.

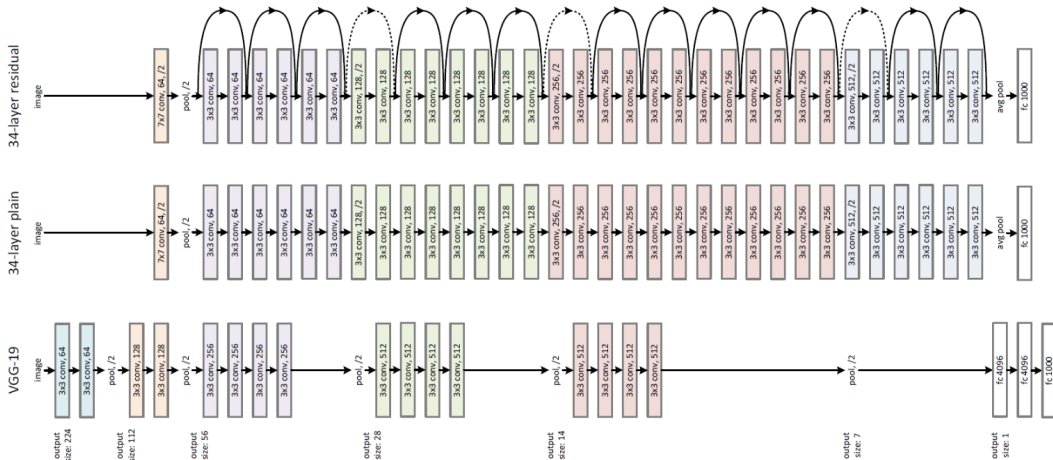
The activation at the output layer with residual connection from layer ℓ :

$$\mathbf{z}^{\{L\}} = \mathbf{z}^{\{\ell\}} + \phi(\mathbf{z}^{\{L-1\}}\mathbf{W}^{\{L\}}) = \mathbf{z}^{\{\ell\}} + \tilde{\mathbf{z}}^{\{L\}}$$

Gradient for weights at layer ℓ :

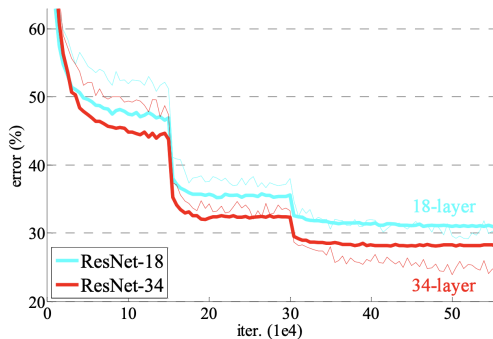
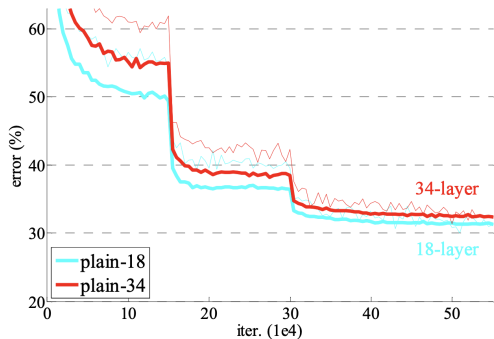
$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{\{\ell\}}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{\{\ell\}}} \frac{\partial \mathbf{z}^{\{\ell\}}}{\partial \mathbf{W}^{\{\ell\}}} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{\{L\}}} \frac{\partial \mathbf{z}^{\{L\}}}{\partial \mathbf{z}^{\{\ell\}}} \frac{\partial \mathbf{z}^{\{\ell\}}}{\partial \mathbf{W}^{\{\ell\}}} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{\{L\}}} \left(1 + \frac{\partial \tilde{\mathbf{z}}^{\{L\}}}{\partial \mathbf{z}^{\{\ell\}}} \right) \frac{\partial \mathbf{z}^{\{\ell\}}}{\partial \mathbf{W}^{\{\ell\}}} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{\{L\}}} \frac{\partial \mathbf{z}^{\{\ell\}}}{\partial \mathbf{W}^{\{\ell\}}} + \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{\{L\}}} \frac{\partial \tilde{\mathbf{z}}^{\{L\}}}{\partial \mathbf{z}^{\{\ell\}}} \frac{\partial \mathbf{z}^{\{\ell\}}}{\partial \mathbf{W}^{\{\ell\}}}\end{aligned}$$

A well known residual network (ResNet) from MSR for image recognition



We will learn the details of the ResNet in Module 5.3.

ResNet outperforms plain network on image recognition

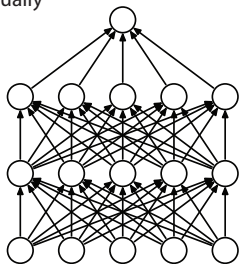


He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).

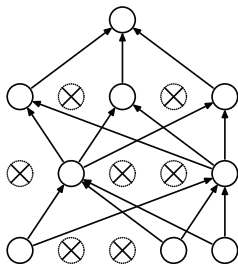
<https://arxiv.org/pdf/1512.03385.pdf>

Regularization by dropout (Murphy22 13.5.4)

Conceptually

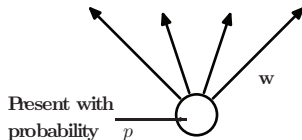


(a) Standard Neural Net

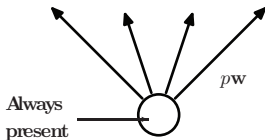


(b) After applying dropout.

In practice



(a) At training time



(b) At test time

During training:

- Drop hidden units with some probability π .
- Train sub-network by back-propagation as usual.

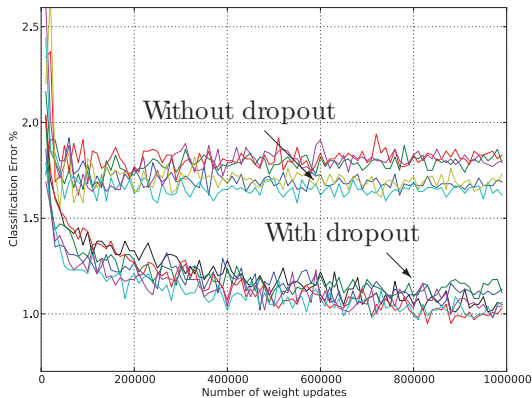
During testing:

- Multiply network weight by the fraction p_j the hidden j being used: $\mathbf{w}_j^{\{\ell\}} = p_j \hat{\mathbf{w}}_j^{\{\ell\}}$

Connection with [Bayesian neural network](#) (Neal 1996) which seeks to integrate out *all* neural network weights:

$$p(y|x, \mathcal{D}) = \int p(y|x, \theta) p(\theta) d\theta$$

Dropout training – results on MNIST digit classification



- The DNN without dropout reaches plateau at much earlier epochs.
- Validation error continues to decrease with dropout even after 600K epochs.
- As a result, training with dropout takes a little longer to converge.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014).

Dropout: a simple way to prevent neural networks from overfitting. The journal of machine learning research, 15(1), 1929-1958.

Bayesian optimization to automatically choose network hyperparameters

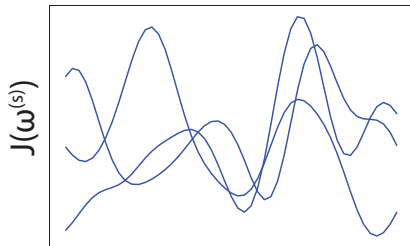
Basic idea: Use a flexible regression model, i.e., Gaussian Process (GP) to **predict** the prediction error $J(\omega)$ as a function of hyperparameters ω (e.g., number of hidden units $K^{\{\ell\}}$ per layer ℓ , number of hidden layers L , weight penalty λ , initial learning rate, momentum, etc):

1. Train a set of neural networks based on an initial set of hyperparameter setting to produce $\mathcal{S} = \{\omega^{(s)}, J(\omega)^{(s)}\}_{s=1}^S$ on the validation data
2. Fit a GP using $\mathcal{S} = \{\omega^{(s)}, J(\omega)^{(s)}\}_{s=1}^S$ as the training set
3. Run GP to evaluate on all of the hyperparameter settings Ω
4. Choose the best setting $\omega^* \in \Omega$ with the *lowest* expected error, train the MLP with the hyperparameter setting ω^* and evaluate it on the validation set to get $J(\omega^*)$, and add the pair to the existing set $\mathcal{S}^* \leftarrow \mathcal{S} \cup \{\omega^*, J(\omega)^*\}$
5. Repeat 2-4 with the new training set $\mathcal{S} = \mathcal{S}^*$

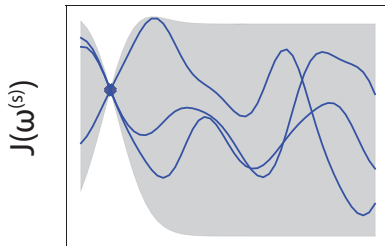
Reference: Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical bayesian optimization of machine learning algorithms. Advances in neural information processing systems, 25.

GP fitting on $\mathcal{S} = \{\omega^{(s)}, J(\omega)^{(s)}\}_{s=1}^S$ (source & Murphy22 Fig. 17.7)

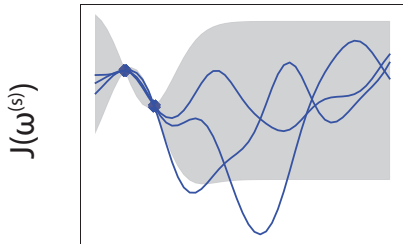
S=0



S=1

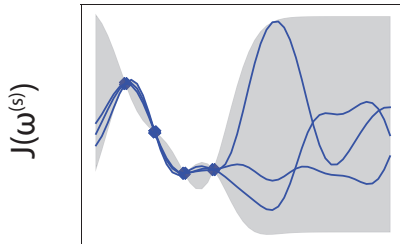


S=2



$\omega^{(s)}$

S=4



$\omega^{(s)}$

Summary

- Calculate the gradients using backpropagation on 1-layer, 2-layer, L-layer MLP for: binary, multi-class, multi-label, and multi-output regression
- Automatic differentiation allows programming deep neural networks without manually deriving the gradients for backprop
- Practical survey of the techniques used to train DNN:
 - parallel training
 - non-saturating activation functions
 - residual connection
 - dropout regularization
 - Bayesian optimization to select hyperparameters